

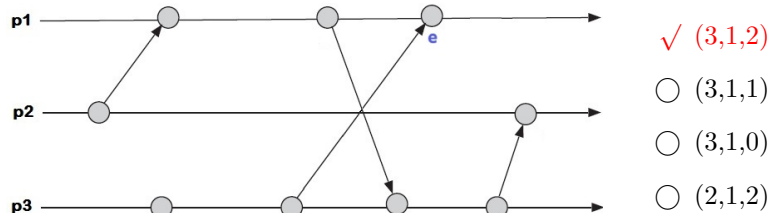
Sistemes Distribuïts en Xarxa (SDX)
Facultat d'Informàtica de Barcelona
Control parcial. 8 d'Abril 2024

Contesteu a les preguntes de manera concisa i precisa
Contesteu al mateix full
Test, seminaris: no es poden consultar apunts
Readings: podeu portar els vostres informes
Problemes: podeu portar un full de notes imprès a doble cara
Durada: 1 hora, 50 minuts

Nom i Cognoms:

1. (5 pts) Seleccioneu la resposta (una només) que considereu correcta en cadascun dels apartats. Cada resposta correcta val 1/2 punts. Cada resposta incorrecta resta 1/6.
- (a) Les trames del protocol Ethernet inclouen un conjunt de bits addicionals al final, anomenat *Frame Check Sequence*, que s'utilitzen per verificar la integritat de la informació rebuda. Quin tipus de redundància s'està aplicant?
- ☒ Information redundancy
 - ☐ Time redundancy
 - ☐ Physical redundancy
 - ☐ Cap, perquè diem que s'usa redundància només quan es repliquen processos
- (b) Si estem mantenint una trucada telefònica mitjançant VoIP (*Voice over IP*) i la veu s'entretalla perquè alguns paquets es reben amb retard, quin tipus de fallada estem patint?
- ☐ Crash failure
 - ☐ Omission failure
 - ☒ Timing failure
 - ☐ Arbitrary (Byzantine) failure
- (c) Quin és el comportament de l'emissor en una comunicació síncrona del tipus *response-based*?
- ☐ L'emissor continua amb la seva execució immediatament després d'enviar la seva petició
 - ☐ L'emissor es bloqueja fins que rep una confirmació de que el middleware es farà càrrec de transmetre la seva petició
 - ☐ L'emissor es bloqueja fins que rep una confirmació de que el receptor ha rebut la seva petició
 - ☒ L'emissor es bloqueja fins que el receptor ha tractat la seva petició i ha rebut la resposta corresponent
- (d) En quina situació es pot generar una computació orfe (*orphan computation*) durant una RPC?
- ☐ Quan es perd el missatge de petició del client
 - ☒ Quan el client té una fallada de crash durant el processament de la RPC
 - ☐ Quan el servidor té una fallada de crash durant el processament de la RPC
 - ☐ Quan es perd el missatge de resposta del servidor

- (e) Un servidor emet un vídeo per streaming amb una cadència de 2 Mbps (megabits per segon) i un client el reproduïx amb una cadència de 800 kbps. En un moment anterior, el client i el servidor van acordar pausar la transmissió perquè el buffer del client estava a punt de desbordar-se. Si la *low-water mark* està fixada a 0.75 MB, quants segons es pot trigar com a màxim en reprendre la recepció de dades des del moment en què el client li demana al servidor per tal que el client no es quedi sense dades disponibles?
- ☐ 0.9375 segons
- ☐ 3 segons
- ☐ 5 segons
- ☒ 7.5 segons
- (f) Un conjunt de nodes resincronitzen el seu rellotge en intervals de 30 minuts per tal que la diferència entre dos rellotges qualsevol no superi els 90 ms. Quina deriva màxima de la freqüència del rellotge respecte al temps perfecte (*maximum clock drift rate*) es pot suportar?
- ☐ 2.5×10^{-2}
- ☐ 1×10^{-5}
- ☒ 2.5×10^{-5}
- ☐ 5×10^{-5}
- (g) Donada la següent seqüència d'events executada pels processos $p1$, $p2$ i $p3$, quin seria el valor del rellotge lògic vectorial de l'event e ?



- (h) Tenim un grup de 6 processos amb identificadors P1–P6 organitzats en sentit horari en un anell lògic. Si P4 inicia una elecció mitjançant l'algorisme de *Chang & Roberts* i la latència entre qualsevol parell de processos és de 10 ms, quant temps trigarà en acabar l'elecció? (assumeix que no hi ha fallades, una única elecció en curs, i temps de processament dels missatges negligible)
- ☐ 80 ms
- ☐ 120 ms
- ☒ 140 ms
- ☐ 180 ms
- (i) Tenim un grup de processos que es comuniquen mitjançant multicast fiable bàsic (*basic reliable multicast*) en el marc d'un sistema distribuït asíncron. Per prevenir el desbordament del seu *history buffer*, en quin moment esborrarà un missatge multicast un procés emissor?
- ☐ Mai, l'emissor ha de mantenir els missatges indefinidament al buffer per satisfer una petició eventual de retransmissió
- ☒ Quan tots els processos hagin contestat l'emissor amb les confirmacions ACK corresponents al missatge multicast
- ☐ Quan hagi passat un temps des de l'enviament superior al temps de *round-trip* màxim del sistema, el qual és conegut
- ☐ Quan hagi passat un temps des de l'enviament superior al temps de *round-trip* estimat del sistema, tot i que això pugui causar que alguna petició de retransmissió no es pugui atendre
- (j) Quin dels següents conjunts (representats com a *version vectors*) no és necessari per implementar els 4 models de consistència centrats en el client (*client-centric consistency models*)?
- ☒ El conjunt d'operacions de lectura executades per un client
- ☐ El conjunt d'operacions d'escriptura executades per un client
- ☐ El conjunt d'operacions d'escriptura que determinen el resultat de les operacions de lectura executades per un client
- ☐ El conjunt d'operacions d'escriptura aplicades per una rèplica fins el moment

Nom i Cognoms:

2. (SEMINARIS) Contesta les següents preguntes referents al seminari **Muty**.

- a) (6 pts) Completa el següent extracte de codi corresponent a la implementació del mecanisme d'exclusió mútua de Ricart & Agrawala utilitzant rellotges lògics de Lamport (i.e. *lock3*).

```
init(MyId) ->
  receive
    {peers, Nodes} ->
      open(MyId, Nodes, 0);
  end.

open(MyId, Nodes, MyClock) ->
  receive
    {take, Master, Ref} ->
      MyNewClock = MyClock + 1,
      Refs = lists:map(fun(P) ->
        R = make_ref(),
        P ! {request, self(), R, MyId, MyNewClock},
        R
      end, Nodes),
      wait(MyId, Nodes, Master, Refs, [], Ref, MyNewClock, MyNewClock);
    {request, From, Ref, _FromId, ReqTime} ->
      MyNewClock = max(MyClock, ReqTime),
      From ! {ok, Ref},
      open(MyId, Nodes, MyNewClock)
  end.
```

- b) (2 pts) El següent extracte de codi inicialitza un worker com a part d'un sistema d'exclusió mútua. Fes els canvis necessaris per executar aquest worker en una instància Erlang remota 'n1@127.0.0.1'.

```
start(Lock, Sleep, Work) ->
  register(w1, worker:start("SDX1", self(), Lock, 1, Sleep, Work)).

stop() ->
  w1 ! stop.

start(Lock, Sleep, Work) ->
  spawn('n1@127.0.0.1',
    fun() -> register(w1, worker:start("SDX1", self(), Lock, 1, Sleep, Work))
  end).

stop() ->
  {w1, 'n1@127.0.0.1'} ! stop.
```

- c) (2 pts) Indica quin comportament no desitjat es pot produir en la implementació del *lock1* i explica per què és possible que es generi aquesta situació en el seu codi.

Solution: *lock1* can cause deadlocks because any process receiving a **request** message when in the WAITING state will always queue the requesting process. As a consequence, it is possible that several processes end up waiting for each other.

3. (SEMINARIS) Contesta les següents preguntes referents als seminaris **Ordy** i **Chatty**.

- a) (6 pts) **Ordy**. Completa el següent extracte de codi corresponent a la implementació de multicast causalment ordenat.

```
% Check if a message from process Id with clock MsgVC can be delivered to the worker
% Invoked initially as checkMsg(Id, MsgVC, VC, size(VC))
checkMsg(_, _, _, 0) -> ready;
checkMsg(Id, MsgVC, VC, Id) ->
    if (element(Id, MsgVC) == element(Id, VC) + 1) ->
        checkMsg(Id, MsgVC, VC, Id-1);
        true -> wait
    end;
checkMsg(Id, MsgVC, VC, N) ->
    if (element(N, MsgVC) =< element(N, VC)) ->
        checkMsg(Id, MsgVC, VC, N-1);
        true -> wait
    end.
```

- b) (2 pts) **Ordy**. Justifica per què una implementació modificada del mòdul *total* per tal que proporcioni també una ordenació FIFO garantiria també una ordenació causal com a conseqüència.

Solution: We prove it by induction from the basic cases of the happened-before relation:

- Process p sends $m1$ and then $m2$. As the multicast is FIFO-ordered, all the processes will deliver $m1$ before $m2$.
- Process p sends $m1$, process q delivers $m1$ and then sends $m2$. Process q will propose for $m2$ a sequence number which is greater than the agreed number for $m1$, therefore the agreed number for $m2$ will certainly be also greater, and all the processes will deliver $m1$ before $m2$.

- c) (2 pts) **Chatty**. Si tots els clients del chat estan connectats a un únic servidor, és possible que un missatge $m2$ enviat per un client B amb posterioritat a un missatge $m1$ enviat per un client A sigui rebut abans per tots els altres clients del chat? I només per alguns clients? Justifica la resposta.

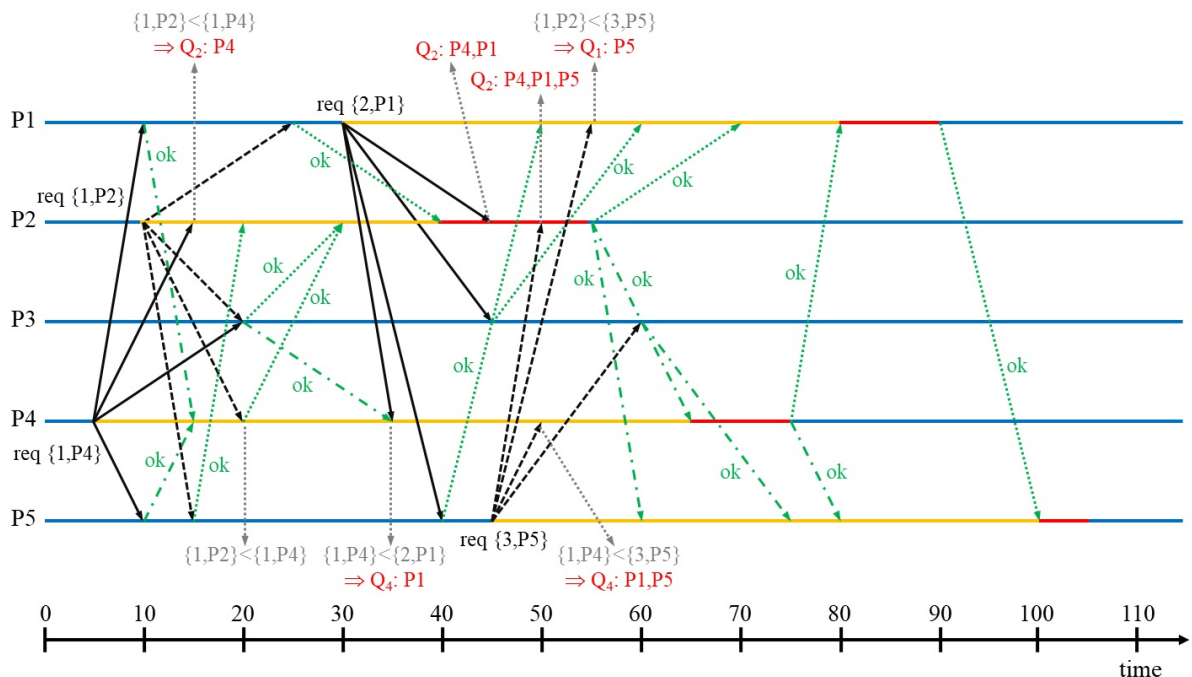
Solution: Due to network latencies, message $m2$ can arrive at the server before message $m1$. In that case, server will send $m2$ and then $m1$, and because of the FIFO ordering guarantee provided by Erlang, all the clients will receive $m2$ before $m1$. It is not possible that only some clients receive $m2$ before $m1$.

Nom i Cognoms:

4. (1.5 pts) Tenim un conjunt de 5 processos ($P1-P5$) que coordinen els seus accessos a una regió crítica mitjançant l'algorisme de *Ricart & Agrawala*. $P1$, $P2$, $P4$ i $P5$ volen accedir a la regió crítica de manera concurrent. La taula de l'esquerra detalla per cada procés en quin instant de temps demana accedir a la regió crítica (columna 1) i quant temps treballa dins de la regió crítica abans d'alliberar-la (columna 2). La taula de la dreta especifica les latències de comunicació entre qualsevol parell de processos (són simètriques). NOTA: Si dos peticions per accedir a la regió crítica tenen el mateix temps lògic, s'utilitza l'identificador del procés (1, 2, 3, ...) per desfer l'empat, essent més prioritari l'identificador més baix.

	temps petició accés	temps dins regió crítica
$P1$	30	10
$P2$	10	15
$P4$	5	10
$P5$	45	5

	$P1$	$P2$	$P3$	$P4$	$P5$
$P1$	0	15	15	5	10
$P2$	15	0	10	10	5
$P3$	15	10	0	15	15
$P4$	5	10	15	0	5
$P5$	10	5	15	5	0



- a) Ajuda't del cronograma per calcular en quin instant accedirà cada procés a la regió crítica.

Solution:

$P1$ accesses the critical section at $t = 80$.

$P2$ accesses the critical section at $t = 40$.

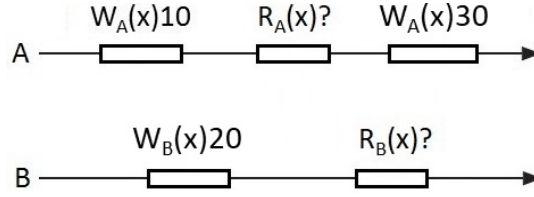
$P4$ accesses the critical section at $t = 65$.

$P5$ accesses the critical section at $t = 100$.

- b) Si un procés allibera la regió crítica i hi ha un únic procés esperant per accedir, indica quin seria el seu *synchronization delay*, mesurat com el nombre de latències de missatge necessàries per què aquest pugui accedir a la regió crítica.

Solution: The synchronization delay will be 1, as granting the waiting process access to the critical section only requires sending one *ok* message more.

5. (1.75 pts) Donats dos clients A i B que executen les següents operacions sobre un objecte replicat x (amb valor inicial 0) tal com es mostren a la figura, indica quin possibles valors poden retornar les operacions $R_A(x)?$ i $R_B(x)?$ si aquesta execució satisfà els següents models de consistència:



- a) Consistència seqüencial

Solution: $R_A(x)?$ can return 10 or 20, but cannot return 0 or 30 because that would violate the program order of Client A.
 $R_B(x)?$ can return 10, 20, or 30, but cannot return 0 because that would violate the program order of Client B.

- b) Linearizability

Solution: $R_A(x)?$ must return the value written by the most recent non-overlapping write operation. As $W_A(x)10$ and $W_B(x)20$ overlap, this value may be 10 or 20.
 $R_B(x)?$ starts after the termination of $R_A(x)?$, hence, it should read the same value (10 or 20), but as it also overlaps with $W_A(x)30$, it may also return 30.

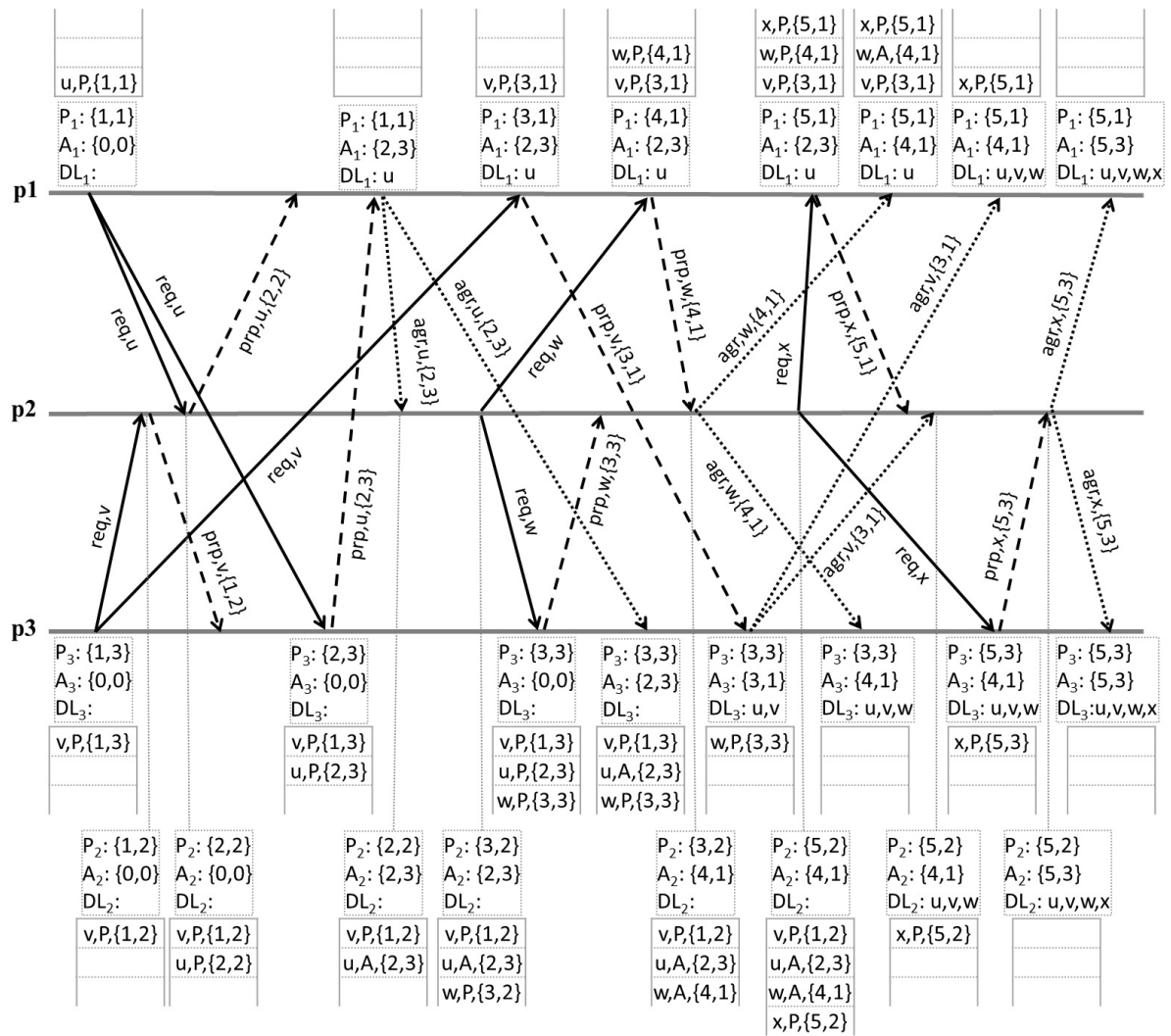
Si les operacions anteriors s'executen en un *data store* de 2 rèpliques ($r1$ i $r2$) que proporciona consistència eventual (sense garanties de sessió pels clients) i que utilitza una política de resolució de conflictes basada en *dotted version vectors*, i tenint en compte que cada operació interactua amb una única rèplica, explica com es duran a terme les següents operacions si tenen lloc en l'ordre indicat (utilitza l'operació d'exemple com a guia). NOTA: Les operacions d'escriptura d'un client inclouen el context causal de la seva darrera operació de lectura, en cas que n'hi hagi alguna.

- a) $W_A(x)10$ a la rèplica $r2$
 $VV_{inc}(\{\}) \geq VV_{r2}(\{\})$
 increment $VV_{inc}[r2]$ and apply 'dot': $\{(r2, 0, 1)\}$
 overwrite x at $r2$: value = 10, $VV = \{(r2, 0, 1)\}$
- b) Propagar actualitzacions de rèplica $r2$ a $r1$
 $VV_{r2}(\{(r2, 0, 1)\}) \geq VV_{r1}(\{\})$
 overwrite x at $r1$: value: 10, $VV = \{(r2, 0, 1)\}^\dagger$
- c) $W_B(x)20$ a la rèplica $r2$
 $VV_{inc}(\{\}) \not\geq VV_{r2}(\{(r2, 0, 1)\})$
 merge(VV_{inc} , VV_{r2}): $\{(r2, 0, 1)\}$
 increment VV and apply 'dot' (with causal context from VV_{inc}): $\{(r2, 0, 2)\}$
 create 'sibling' of x at $r2$: value = 20, $VV = \{(r2, 0, 2)\}$
- d) $R_A(x)?$ a la rèplica $r1$
 Read x at $r1$: value = 10, $VV = \{(r2, 0, 1)\}$
- e) $R_B(x)?$ a la rèplica $r2$
 Read x at $r2$: *sibling*₁ value = 10, $VV = \{(r2, 0, 1)\}$; *sibling*₂ value = 20, $VV = \{(r2, 0, 2)\}$
- f) $W_A(x)30$ a la rèplica $r1$
 $VV_{inc}(\{(r2, 0, 1)\}) \geq VV_{r1}(\{(r2, 0, 1)\})$
 increment $VV_{inc}[r1]$ and apply 'dot': $\{(r1, 0, 1), (r2, 1)\}$
 overwrite x at $r1$: value = 30, $VV = \{(r1, 0, 1), (r2, 1)\}$

[†]Dotted version vectors do not increment the version number at the destination replica when synchronizing, but that would be a valid alternative solution.

Nom i Cognoms:

6. (1.75 pts) Donat un grup de processos que es comuniquen mitjançant la variant de multicast totalment ordenat (*totally-ordered multicast*) en què els processos decideixen de manera coordinada els números de seqüència dels missatges, completa la següent la figura indicant per cada procés i : el contingut dels missatges que envia (els tipus de missatges són els següents: *request for number proposals (req)*, *number proposal (prp)*, *agreed number (agr)*), els missatges guardats a la *hold-back queue* pendents de ser lliurats (indicant el seu número de seqüència i si aquest està acordat (A) o només proposat (P)), la llista de missatges lliurats fins el moment (DL_i), i el valor de les variables d'estat de cada procés en els punts indicats, assumint que A_i fa referència al número de seqüència acordat més gran que el procés i ha rebut fins aquell moment i P_i fa referència al número de seqüència més gran que el procés i ha proposat fins aquell moment. Tingues en compte que el procés emissor d'un missatge multicast també participa en l'algorisme per acordar el seu número de seqüència encara que els missatges que s'envia a si mateix no es veuen a la figura. NOTA: Els números de seqüència són tuples que també inclouen l'identificador del procés, el qual es fa servir per resoldre empats quan els números de seqüència són iguals.



7. (READINGS) Contesta les següents preguntes referents als articles llegits a l'assignatura.

- i) Explica els dos aspectes que es consideren en l'assumpció errònia 'Transport Cost Is Zero' segons es descriu a l'article *Hebert13*.

Solution: The first one relates to the cost of transporting data in terms of time (e.g., larger data structures take longer to be serialized than small ones), and the second one is related to the cost of transporting data in terms of money (e.g., network bandwidth is often more expensive than memory/disk).

- ii) Explica quin és el *trade-off* a que s'enfronta un programador quan ha de decidir quina rutina invocar per obtenir el temps segons es descriu a l'article *Neville-Neil16*.

Solution: The trade-off that must be made when deciding how to get the time in a program is between speed and accuracy: the more precise you want the time to be, the longer it will take to retrieve the time.

- iii) Indica quin serà el comportament d'una operació de lectura per un client que no ha realitzat cap operació d'escriptura si es proporciona un model de consistència *Read My Writes* segons es descriu a l'article *Terry13*.

Solution: For clients that have issued no writes, the *Read My Writes* guarantee is the same as *Eventual Consistency*, that is, a read on an object can return any value that was written on that object in the past.